

Стать IT-шником может каждый



Тип данных pandas. DataFrame



Преподаватель



Ваше фото

ИМЯ ФАМИЛИЯ

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)



ВАЖНО!

- Камера должна быть включена на протяжении всего занятия.
- Если у Вас возник вопрос в процессе занятия, пожалуйста, поднимите руку и дождитесь, пока преподаватель закончит мысль и спросит Вас, также можно задать вопрос в чате или когда преподаватель скажет, что начался блок вопросов.
- Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях.
- Вести себя уважительно и этично по отношению к остальным участникам занятия.
- Во время занятия будут интерактивные задания, будьте готовы включить микрофон или демонстрацию экрана по просьбе преподавателя.

ПЛАН ЗАНЯТИЯ

Введение

Основной блок

Задание для закрепления

Вопросы по основному блоку

Заключение



Стать IT-шником может каждый



ПОВТОРЕНИЕ ИЗУЧЕННОГО





Повторение

- Библиотека pandas
- Тип данных Series
- Индексация
- Продвинутые операции с Series



Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Введение

Создание DataFrame

Анализ DataFrame

Индексация в DataFrame



Стать IT-шником может каждый



ОСНОВНОЙ БЛОК



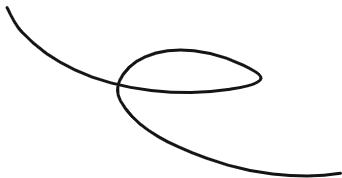
Создание DataFrame





DataFrame —

это двумерная таблица с данными. Имеет индекс и набор столбцов (возможно, имеющих разные типы). При этом и каждый столбец, и каждая строка будут Series.



Создание DataFrame

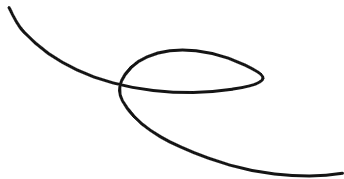
```
d = {'one': pd.Series(range(6), index=list('aaefg'))}  
df = pd.DataFrame(d)  
df
```

```
#      one  
# a      0  
# a      1  
# a      2  
# e      3  
# f      4  
# g      5
```



display: display(df) —

это инструкция, которую используют для визуализации датафрейма в jupyter notebook / colab.



Сброс индекса

```
df.reset_index()
#   index one
#0     a   0
#1     a   1
#2     a   2
#3     e   3
#4     f   4
#5     g   5
```


Датафрейм с несколькими столбцами

```
d = {'one': pd.Series(range(6), index=list('abdefg')),  
      'two': pd.Series(range(7), index=list('abcdefg')),  
      'three': pd.Series(np.random.normal(size=7), index=list('abcdefg'))}
```

```
df = pd.DataFrame(d)
```

```
df
```

#	one	two	three
#a	0.0	0	-1.260839
#b	1.0	1	-0.380510
#c	NaN	2	1.370570
#d	2.0	3	0.302637
#e	3.0	4	1.690632
#f	4.0	5	-0.757313
#g	5.0	6	0.560045

Датафрейм с несколькими столбцами

```
df['one']  
#a  0.0  
#b  1.0  
#c  NaN  
#d  2.0  
#e  3.0  
#f  4.0  
#g  5.0  
#Name: one, dtype: float64
```

Датафрейм с несколькими столбцами

```
df.index  
#Index(['a', 'b', 'c', 'd', 'e', 'f', 'g'], dtype='object')
```

Таблица с несколькими разными типами данных

```
df2 = pd.DataFrame({ 'A': 1.,
                      'B': pd.Timestamp('20130102'),
                      'C': pd.Series(1, index=list(range(4)),
                                     dtype='float32'),
                      'D': np.array([3] * 4,
                                     dtype='int32'),
                      'E': pd.Categorical(["test", "train",
                                           "test", "train"]), # one - hot encoding
                      'F': 'foo' })
```

#	A	B	C	D	E	F
#0	1.0	2013-01-02	1.0	3	test	foo
#1	1.0	2013-01-02	1.0	3	train	foo
#2	1.0	2013-01-02	1.0	3	test	foo
#3	1.0	2013-01-02	1.0	3	train	foo

Таблица с несколькими разными типами данных

```
df2.dtypes  
#A      float64  
#B  datetime64[ns]  
#C      float32  
#D       int32  
#E     category  
#F       object  
#dtype: object
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



Анализ DataFrame



Анализ DataFrame

```
df.head() # в аргументе можем указать сколько первых строк хотим вывести  
df.tail(3) # (а здесь последних). по умолчанию – 5  
df.index # индексы  
df.columns # названия колонок
```

Анализ DataFrame

```
df.values # получение обычной матрицы данных
#array([[ 0.        ,  0.        , -1.26083858],
#       [ 1.        ,  1.        , -0.38050995],
#       [ nan,  2.        ,  1.37057007],
#       [ 2.        ,  3.        ,  0.30263747],
#       [ 3.        ,  4.        ,  1.69063229],
#       [ 4.        ,  5.        , -0.75731332],
#       [ 5.        ,  6.        ,  0.56004507]])
```

Описательные статистики

```
df.describe()
```

Транспонирование данных

```
df.T
#      a      b      c      d      e      f      g
#one  0.0000  1.0000 NaN    2.0000  3.0000  4.0000  5.0000
#two  0.0000  1.0000  2.0000  3.0000  4.0000  5.0000  6.0000
#three -1.261 -0.380  1.3705  0.3026  1.6906 -0.757  0.5600
```

Сортировка по столбцу

```
df.sort_values(by='three', ascending=False)
```

Итерирование по столбцам

```
for i in df.columns:  
    print(i, df[i])
```


Итерирование по строкам

```
for i, j in df.iterrows():  
    print(i, j)
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ЗАДАНИЕ ДЛЯ ЗАКРЕПЛЕНИЯ





Задание:

Сгенерируйте массив точек в 3D, создайте по нему датафрейм и отсортируйте строки в порядке следования осей.





Задание:

Сгенерируйте массив точек в 3D, создайте по нему датафрейм и отсортируйте строки в порядке следования осей.

```
pd.DataFrame(  
    np.random.normal(size=(100, 3)),  
    columns=['x', 'y', 'z']  
) .sort_values(by=['x', 'y', 'z'])
```

Стать IT-шником может каждый



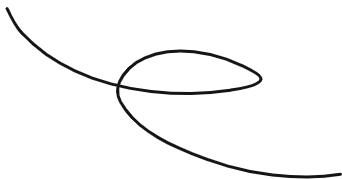
Индексация в DataFrame





.loc, .iloc —

это атрибуты, которые используются для написания продуктивного кода при обработке большого объема данных.



Индексация в DataFrame

```
df['one'] # Если в качестве индекса указать имя столбца, получится одномерный набор данных типа Series.  
df.one   # К столбцу можно обращаться как к полю объекта, если имя столбца позволяет это сделать (= является строкой)  
df['one'].index # Индексы полученного одномерного набора данных.  
df['one'].name  # Можно получить имя данного столбца  
df['one']['d']  # Получение элемента массива. Сначала надо указать столбец, потом строку  
df['c']['one']  # А наоборот нельзя!
```


Правила индексации в pandas

```
df['b':'d'] # правая граница включена!  
df[1:3] # но диапазон целых чисел даёт диапазон строк с такими номерами, не включая правую границу (как  
обычно при индексировании списков).  
df[1] # при этом обратиться к индексу по номеру нельзя, а вот срез взять можно!
```

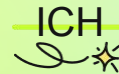


```
df[['one', 'three']]  
display(df['one'])  
display(df[['one']]) # как вы думаете, чем обусловлено отличие в представлении этой и прошлой строки?
```

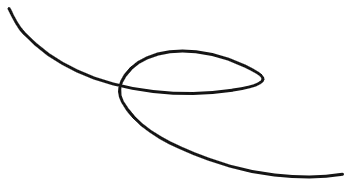
Атрибут loc

```
df.loc['b'] # __index__  
df.loc['b', 'one']  
df['one']['b'] # эта строка выведет то же что и предыдущая  
df.loc['a':'b', 'one'] # можно использовать срезы по-всякому  
df.loc['a':'b', :]  
df.loc[:, 'one'] # двоеточие, напомним, означает, что берем все значения по соответствующей размерности  
df.iloc[2] # атрибут iloc подобен loc: первый индекс — номер строки, второй — номер столбца. Это целые числа,  
конец диапазона не включается как обычно в питоне  
df.iloc[1:3]  
df.iloc[1:3, 0:2] # и тут срезы абсолютно валидны
```

Полезно знать



Булевская индексация —
это **выбор строк** с заданным условием.



Булевская индексация

```
df[df.three > 0]  
df[df.notna()]
```

Стать IT-шником может каждый



ВОПРОСЫ



Стать IT-шником может каждый



ПРАКТИЧЕСКАЯ РАБОТА



Практическое задание

Сгенерируйте случайную целочисленную матрицу $n \times m$, где $n=20, m=10$. Создайте из нее датафрейм, пронумеровав столбцы случайной перестановкой чисел из $\{1, \dots, m\}$. Выберите столбцы с четными номерами и строки, в которых четных элементов больше, чем нечетных.

Стать IT-шником может каждый



ОСТАВШИЕСЯ ВОПРОСЫ





Домашнее задание

1. Напишите программу, которая:
 - Запрашивает у пользователя ввод чисел, разделенных пробелами.
 - Преобразует введенные данные в объект Series, используя в качестве индексов буквы алфавита начиная с 'a'.
 - Выводит созданный объект Series на экран.



Домашнее задание

2. Напишите программу, которая:

- Запрашивает у пользователя данные для создания DataFrame. Данные включают в себя имена столбцов и строки значений, вводимые через пробел.
- Создает DataFrame с использованием введенных данных.
- Выводит созданный DataFrame на экран.
- Запрашивает у пользователя имя столбца, по которому необходимо отфильтровать данные.
- Выводит значения указанного столбца.



Стать IT-шником может каждый



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ





Домашнее задание

1. Напишите программу, которая:
 - Генерирует два больших одномерных массива с 1 миллионом элементов каждый, заполненных случайными числами.
 - Сначала выполняет поэлементное сложение этих массивов, используя циклы Python.
 - Затем выполняет ту же операцию, используя возможности NumPy.
 - Измеряет и выводит время выполнения каждого подхода.

Пример вывода:

Время выполнения сложения с использованием цикла Python: 0.5 секунды.

Время выполнения сложения с использованием NumPy: 0.01 секунды.



Домашнее задание

2. Создать большой двумерный массив и исследовать, как изменение порядка (C-style и Fortran-style) влияет на производительность операций сложения и умножения. Можно использовать тот же бенчмарк что и в лекции. Необходимо погуглить про то, как завести массив с порядком в Fortran-style. Не забывайте про magic в jupyter.



Домашнее задание

3. Напишите программу, которая:

- Запрашивает у пользователя размер квадратной матрицы N .
- Генерирует квадратную матрицу размером $N \times N$, заполненную случайными числами от -10 до 10 .
- Выводит сгенерированную матрицу на экран.
- Вычисляет и выводит определитель матрицы.
- Если определитель не равен нулю, программа должна вычислить и вывести обратную матрицу. В противном случае, программа должна сообщить, что обратной матрицы не существует.

Полезные ссылки

- [10 Minutes to pandas](#)
- [Indexing and selecting data — pandas 2.2.1 documentation](#)

Стать IT-шником может каждый



ЗАКЛЮЧЕНИЕ

